**A Case Study Report on**

# "Online Retail Transaction Clustering Using Machine Learning Techniques"

**for the course**

## Fundamentals Of Data Science (22CSC59)

Submitted in partial fulfillment of the requirements for the award of degree

## BACHELOR OF ENGINEERING

**in**

## COMPUTER SCIENCE AND ENGINEERING


**by**

**Munugoti Sneha(160124733152)**

**CSE-III**

**3rd Year**

**Faculty:**

**Smt. S.Durga Devi Assistant Professor**






**Department of Computer Science and Engineering,**

**Chaitanya Bharathi Institute of Technology (Autonomous),**

**(Affiliated to Osmania University, Hyderabad) Hyderabad, TELANGANA (INDIA) –500 075 [2026-2027]**

## 1. PROBLEM STATEMENT

The online retail store maintains transaction records containing products purchased by customers. However, the store lacks a systematic approach to understanding transaction characteristics and identifying groups of similar shopping baskets. This case study aims to analyze retail transaction data using descriptive statistics, data visualization, dimensionality reduction, and clustering algorithms. K-means, hierarchical clustering, DBSCAN, and spectral clustering will be implemented and evaluated using appropriate performance metrics. The findings will help identify transaction patterns and provide insights that may support product recommendations, promotional strategies, and inventory planning.

## 2. METHODOLOGY

1. Load the transaction file and inspect its dimensions and schema.
2. Treat each row as one transaction and each non-empty cell as a product.
3. Normalize product text and remove repeated items within a basket for binary presence/absence analysis.
4. Compute transaction-size and product-frequency statistics.
5. Convert baskets to a binary transaction–product matrix.
6. Use PCA for a two-dimensional visualization.
7. Compare K-means, hierarchical agglomerative clustering, DBSCAN, and spectral clustering with silhouette and Davies–Bouldin scores where valid.
8. State limitations and business implications based on computed results.Figure 1. Loading and Inspecting the Bank Marketing Dataset

```
from pathlib import Path
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN, SpectralClustering
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, davies_bouldin_score

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Update this path if you move the notebook and dataset to another folder.
DATA_PATH = Path("Online retail_new (1).csv")
if not DATA_PATH.exists():
    # Helpful fallback for the original uploaded-file location in this environment.
    DATA_PATH = Path("/mnt/data/Online retail_new (1).csv")

print("Dataset path:", DATA_PATH)
print("Exists:", DATA_PATH.exists())
```

```
Dataset path: Online retail_new (1).csv
Exists: True
```

The imports provide data handling (pandas, numpy), plots (matplotlib), PCA, clustering algorithms, and evaluation metrics. A fixed random seed makes sampling and model initialization reproducible. The path check supports running the notebook either beside the CSV or in the current hosted environment.

```
df = pd.read_csv(DATA_PATH)
print("Rows (transactions):", df.shape[0])
print("Columns (item slots):", df.shape[1])
display(df.head())
print("\\nData types:")
display(df.dtypes.to_frame("dtype"))
print("\\nNon-null values per row (first 10):")
display(df.notna().sum(axis=1).head(10).to_frame("items_in_transaction"))
```

```
Rows (transactions): 7500
Columns (item slots): 20
```

| | shrimp | almonds | avocado | vegetables mix | green grapes | whole weat flour | yams | cottage cheese | energy drink | tomato juice | low fat yogurt | green tea |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | burgers | meatballs | eggs | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| **1** | chutney | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| **2** | turkey | avocado | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| **3** | mineral water | milk | energy bar | whole wheat rice | green tea | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| **4** | low fat yogurt | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |

| honey | salad | mineral water | salmon | antioxydant juice | frozen smoothie | spinach | olive oil |
|---|---|---|---|---|---|---|---|
| NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |

```
\nData types:
```

| | dtype |
|---|---|
| **shrimp** | object |
| **almonds** | object |
| **avocado** | object |
| **vegetables mix** | object |
| **green grapes** | object |
| **whole weat flour** | object |
| **yams** | object |
| **cottage cheese** | object |
| **energy drink** | object |

```
\nNon-null values per row (first 10):
```

| | items_in_transaction |
|---|---|
| **0** | 3 |
| **1** | 1 |
| **2** | 2 |
| **3** | 5 |
| **4** | 1 |
| **5** | 2 |
| **6** | 3 |
| **7** | 3 |
| **8** | 1 |

## Interpretation

Each row is treated as one transaction, while the columns are item slots rather than separate measured features. Blank cells mean that the basket contains fewer products than the maximum row capacity. Because the source has no transaction identifier, customer identifier, timestamp, price, or quantity, the analysis focuses on basket contents only.

```
transactions = []
for _, row in df.iterrows():
    basket = [
        str(value).strip().lower()
        for value in row.tolist()
        if pd.notna(value) and str(value).strip()
    ]
    # A basket is modeled as item presence/absence; duplicate product entries in a row count once.
    transactions.append(sorted(set(basket)))

n_transactions = len(transactions)
basket_sizes = np.array([len(basket) for basket in transactions])
product_counts = Counter(item for basket in transactions for item in basket)
products = sorted(product_counts)

print(f"Transactions: {n_transactions:,}")
print(f"Distinct products observed: {len(products):,}")
print(f"Empty transactions: {sum(size == 0 for size in basket_sizes):,}")
print(f"Duplicate transaction rows (same basket, disregarding item order): {n_transactions -
len(set(tuple(b) for b in transactions)):,}")
print("Missing cells in source table:", int(df.isna().sum().sum()))
print("Example cleaned baskets:", transactions[:5])
```

```
Transactions: 7,500
Distinct products observed: 119
Empty transactions: 0
Duplicate transaction rows (same basket, disregarding item order): 2,347
Missing cells in source table: 120657
Example cleaned baskets: [['burgers', 'eggs', 'meatballs'], ['chutney'], ['avocado', 'turkey'], ['energy
bar', 'green tea', 'milk', 'mineral water', 'whole wheat rice'], ['low fat yogurt']]
```

### Code explanation and cleaning decisions

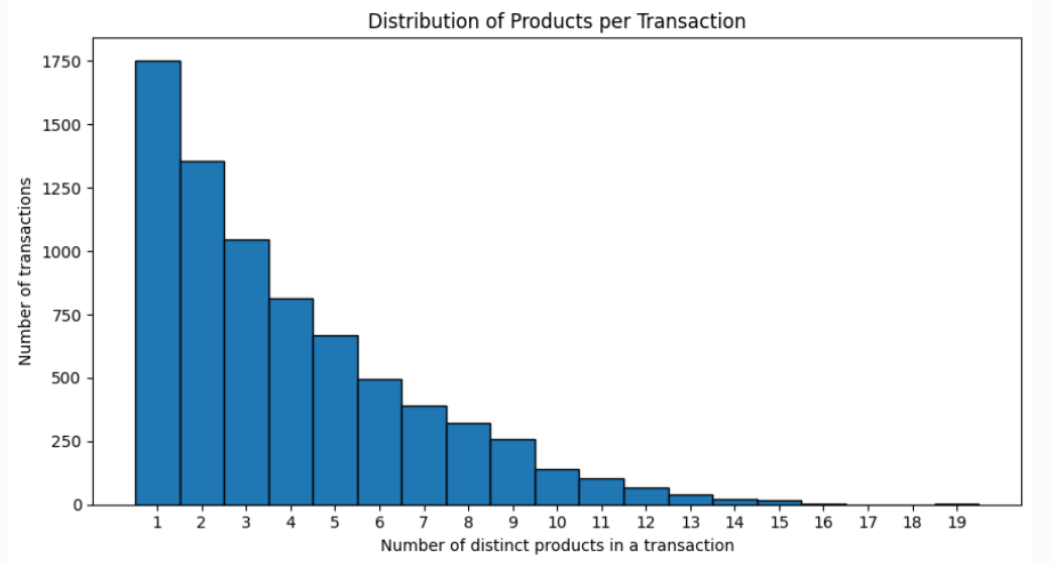
Blank cells are expected in a basket dataset because a transaction may contain fewer items than the widest basket. They are therefore not imputed. Product labels are trimmed and lowercased to avoid treating simple capitalization or whitespace differences as separate products. Repeated items inside a single row are removed for binary basket analysis. Duplicate baskets are reported, not automatically deleted: two different purchases can legitimately contain the same set of products.

```
size_summary = pd.Series(basket_sizes, name="items_per_transaction").describe()
display(size_summary.to_frame())
print("Median basket size:", float(np.median(basket_sizes)))
print("Most common basket size(s):", pd.Series(basket_sizes).mode().tolist())
```

| | items_per_transaction |
|---|---|
| **count** | 7500.000000 |
| **mean** | 3.911733 |
| **std** | 2.899149 |
| **min** | 1.000000 |
| **25%** | 2.000000 |
| **50%** | 3.000000 |
| **75%** | 5.000000 |
| **max** | 19.000000 |

```
Median basket size: 3.0
Most common basket size(s): [1]
```

```
plt.figure(figsize=(9, 5))
bins = np.arange(0.5, basket_sizes.max() + 1.5, 1)
plt.hist(basket_sizes, bins=bins, edgecolor="black")
plt.title("Distribution of Products per Transaction")
plt.xlabel("Number of distinct products in a transaction")
plt.ylabel("Number of transactions")
plt.xticks(range(1, int(basket_sizes.max()) + 1))
plt.tight_layout()
plt.show()
```




**Statistical finding: basket size**

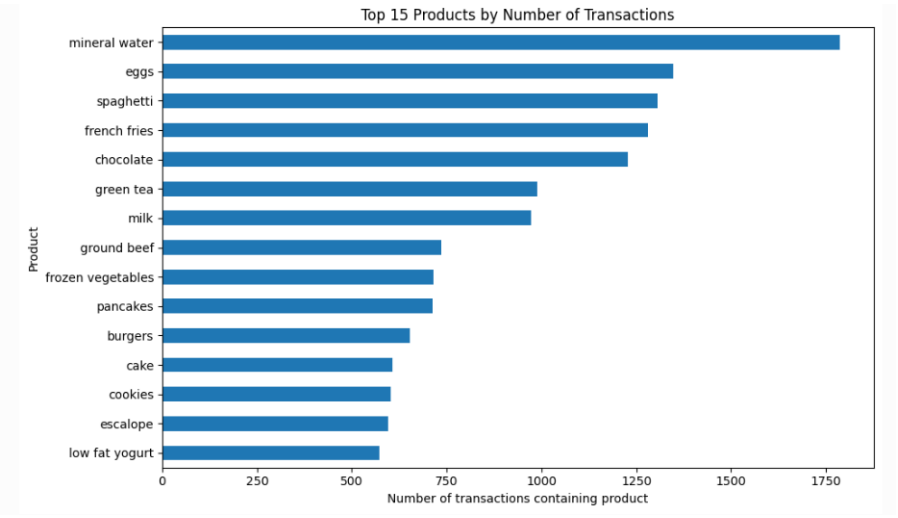
Use the descriptive table and histogram to describe the typical number of distinct products per transaction, the spread, and whether the distribution is concentrated in small baskets or has a long right tail. The plotted counts are transaction counts, not sales revenue. Since the

dataset contains no prices or quantities, basket size must not be interpreted as transaction value.

```
top_n = 15
top_products = pd.Series(product_counts).sort_values(ascending=False).head(top_n)
display(top_products.rename("transaction_count").to_frame())

plt.figure(figsize=(10, 6))
top_products.sort_values().plot(kind="barh")
plt.title(f"Top {top_n} Products by Number of Transactions")
plt.xlabel("Number of transactions containing product")
plt.ylabel("Product")
plt.tight_layout()
plt.show()
```

| | transaction_count |
|---|---|
| **mineral water** | 1787 |
| **eggs** | 1348 |
| **spaghetti** | 1306 |
| **french fries** | 1282 |
| **chocolate** | 1229 |
| **green tea** | 990 |
| **milk** | 972 |
| **ground beef** | 737 |
| **frozen vegetables** | 715 |




**Statistical finding: product frequency**

Each bar counts how many transaction baskets contain a product. This is transaction frequency (support count), not units sold. The most frequent products may be candidates for shelf visibility or bundle analysis, but frequency alone does not prove profitability or causation.

```
product_to_col = {product: i for i, product in enumerate(products)}
X = np.zeros((n_transactions, len(products)), dtype=np.uint8)
for row_i, basket in enumerate(transactions):
    for product in basket:
        X[row_i, product_to_col[product]] = 1

X_df = pd.DataFrame(X, columns=products)
print("Binary feature matrix shape:", X.shape)
print("Rows = transactions; columns = products")
display(X_df.head())
print("Overall matrix density (share of 1s):", round(float(X.mean()), 4))
```

```
Binary feature matrix shape: (7500, 119)
Rows = transactions; columns = products
```

| | almonds | antioxydant juice | asparagus | avocado | babies food | bacon | barbecue sauce | black tea | blueberries | body spray | bramble |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **2** | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **3** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **4** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

5 rows × 119 columns

```
Overall matrix density (share of 1s): 0.0329
```

### Code explanation

The matrix uses one row per transaction and one column per distinct product. A value of 1 means the product appears in that basket; 0 means it does not. This representation is appropriate for presence/absence clustering and avoids inventing quantities. It is sparse conceptually because most transactions contain only a small fraction of all possible products.

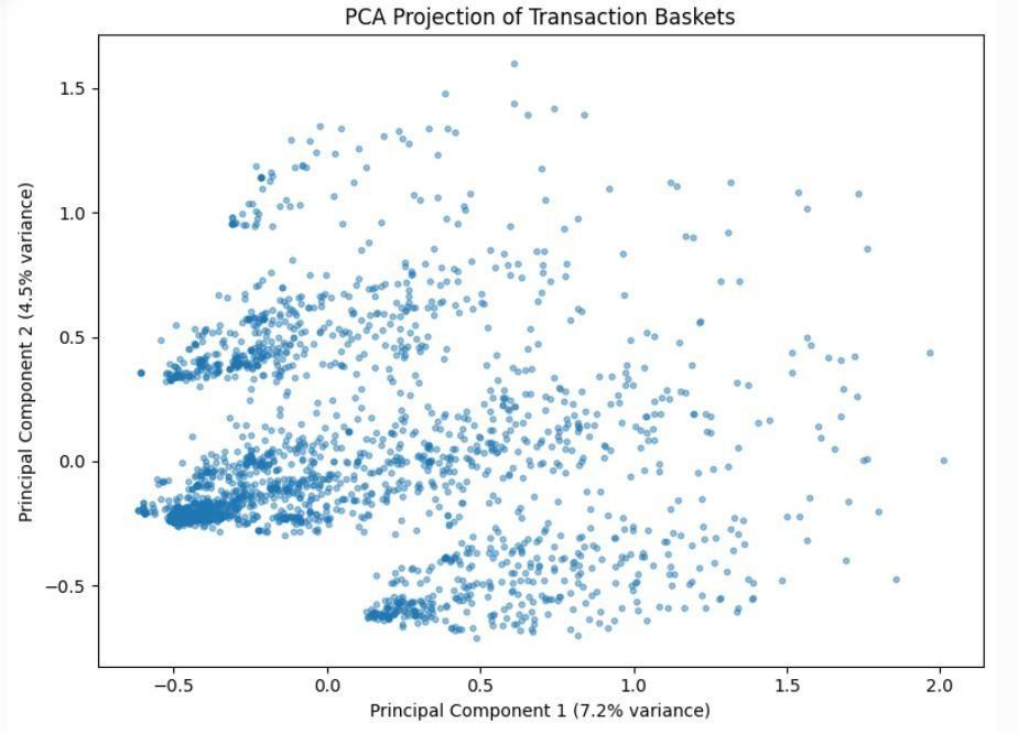
```python
rng = np.random.default_rng(RANDOM_STATE)
pca_n = min(2500, len(X))
pca_idx = np.sort(rng.choice(len(X), size=pca_n, replace=False))
X_pca_input = X[pca_idx].astype(float)

pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_pca_input)
explained = pca.explained_variance_ratio_

print("Variance explained by PC1:", round(float(explained[0]), 4))
print("Variance explained by PC2:", round(float(explained[1]), 4))
print("Combined variance explained:", round(float(explained.sum()), 4))

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], s=10, alpha=0.45)
plt.title("PCA Projection of Transaction Baskets")
plt.xlabel(f"Principal Component 1 ({explained[0]:.1%} variance)")
plt.ylabel(f"Principal Component 2 ({explained[1]:.1%} variance)")
plt.tight_layout()
plt.show()
```

```
Variance explained by PC1: 0.0719
Variance explained by PC2: 0.0445
Combined variance explained: 0.1164
```




**PCA interpretation**

Each point represents one sampled transaction projected into two dimensions. Nearby points have similar projections, but visual overlap does not guarantee identical baskets. The two displayed components retain only the variance printed above, so this chart is an exploratory view rather than proof that clear natural clusters exist.

```
# 7. K-means: choose k using silhouette score
# K-means is fit on up to 2,500 sampled transactions for manageable execution.
km_n = min(2500, len(X))
km_idx = np.sort(rng.choice(len(X), size=km_n, replace=False))
X_km = X[km_idx].astype(float)

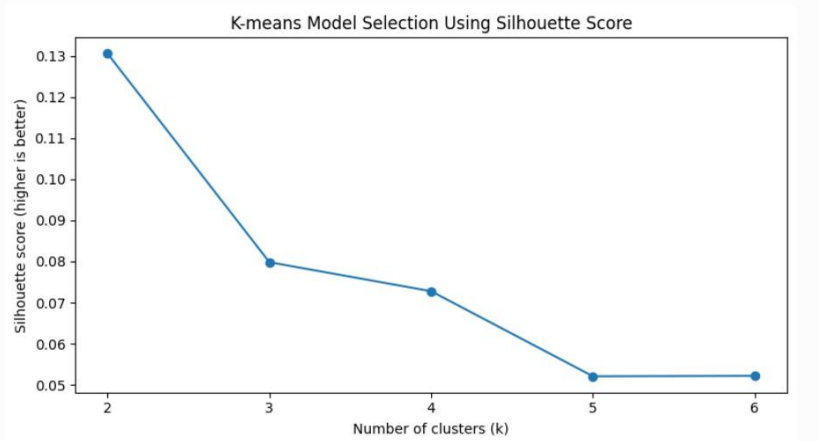
k_rows = []
for k in range(2, 7):
    model = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    labels = model.fit_predict(X_km)
    sil = silhouette_score(X_km, labels, sample_size=min(1000, len(X_km)), random_state=RANDOM_STATE)
    dbi = davies_bouldin_score(X_km, labels)
    k_rows.append({"k": k, "inertia": model.inertia_, "silhouette": sil, "davies_bouldin": dbi})

k_results = pd.DataFrame(k_rows)
display(k_results)
best_k = int(k_results.loc[k_results["silhouette"].idxmax(), "k"])
print("Selected k by highest sampled silhouette score:", best_k)

plt.figure(figsize=(8, 4.5))
plt.plot(k_results["k"], k_results["silhouette"], marker="o")
plt.title("K-means Model Selection Using Silhouette Score")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette score (higher is better)")
plt.xticks(k_results["k"])
plt.tight_layout()
plt.show()
```

| | k | inertia | silhouette | davies_bouldin |
|---|---|---|---|---|
| **0** | 2 | 8547.855808 | 0.130576 | 3.479130 |
| **1** | 3 | 8251.853213 | 0.079829 | 3.321648 |
| **2** | 4 | 8009.183306 | 0.072768 | 3.180475 |
| **3** | 5 | 7838.736824 | 0.052063 | 3.111874 |
| **4** | 6 | 7708.810905 | 0.052164 | 3.062097 |

```
Selected k by highest sampled silhouette score: 2
```




## K-means interpretation

The selected value of *k* is the candidate with the highest silhouette score among k = 2 to 6 on the stated sample. Silhouette values near 1 indicate well-separated groups, values near 0 indicate overlapping groups, and negative values suggest many points may be assigned to the wrong cluster. The Davies–Bouldin index is interpreted in the opposite direction: lower is generally better. This is a limited search over k, not proof of a globally optimal cluster count.

```
comparison = []

# K-means on up to 2,500 rows
km = KMeans(n_clusters=best_k, random_state=RANDOM_STATE, n_init=10)
km_labels = km.fit_predict(X_km)
comparison.append({
    "algorithm": "K-means",
    "sample_size": len(X_km),
    "clusters_excluding_noise": len(set(km_labels)),
    "noise_points": 0,
    "silhouette": silhouette_score(X_km, km_labels, sample_size=min(1000, len(X_km)),
random_state=RANDOM_STATE),
    "davies_bouldin": davies_bouldin_score(X_km, km_labels)
})

# Hierarchical agglomerative clustering on up to 1,000 rows
agg_n = min(1000, len(X))
agg_idx = np.sort(rng.choice(len(X), size=agg_n, replace=False))
X_agg = X[agg_idx].astype(float)
agg = AgglomerativeClustering(n_clusters=best_k, linkage="ward")
agg_labels = agg.fit_predict(X_agg)
comparison.append({
    "algorithm": "Agglomerative",
    "sample_size": len(X_agg),
    "clusters_excluding_noise": len(set(agg_labels)),
    "noise_points": 0,
    "silhouette": silhouette_score(X_agg, agg_labels, sample_size=min(800, len(X_agg)),
random_state=RANDOM_STATE),
    "davies_bouldin": davies_bouldin_score(X_agg, agg_labels)
})
```

```
# DBSCAN on up to 1,000 rows. eps is a baseline choice and may need tuning.
db = DBSCAN(eps=1.0, min_samples=5, metric="euclidean")
db_labels = db.fit_predict(X_agg)
db_clusters = len(set(db_labels) - {-1})
db_noise = int(np.sum(db_labels == -1))
db_sil, db_dbi = np.nan, np.nan
non_noise = db_labels != -1
if db_clusters >= 2 and non_noise.sum() > db_clusters:
    try:
        db_sil = silhouette_score(X_agg[non_noise], db_labels[non_noise],
                                  sample_size=min(800, int(non_noise.sum())), random_state=RANDOM_STATE)
        db_dbi = davies_bouldin_score(X_agg[non_noise], db_labels[non_noise])
    except ValueError:
        pass
comparison.append({
    "algorithm": "DBSCAN",
    "sample_size": len(X_agg),
    "clusters_excluding_noise": db_clusters,
    "noise_points": db_noise,
    "silhouette": db_sil,
    "davies_bouldin": db_dbi
})
```

```
# Spectral clustering on up to 500 rows because it is more computationally expensive.
spec_n = min(500, len(X))
spec_idx = np.sort(rng.choice(len(X), size=spec_n, replace=False))
X_spec = X[spec_idx].astype(float)
spec = SpectralClustering(n_clusters=best_k, random_state=RANDOM_STATE,
                          affinity="nearest_neighbors", n_neighbors=min(10, spec_n-1),
                          assign_labels="kmeans")
spec_labels = spec.fit_predict(X_spec)
comparison.append({
    "algorithm": "Spectral",
    "sample_size": len(X_spec),
    "clusters_excluding_noise": len(set(spec_labels)),
    "noise_points": 0,
    "silhouette": silhouette_score(X_spec, spec_labels,
                                   sample_size=min(400, len(X_spec)), random_state=RANDOM_STATE),
    "davies_bouldin": davies_bouldin_score(X_spec, spec_labels)
})

comparison_df = pd.DataFrame(comparison)
display(comparison_df)
```

| | algorithm | sample_size | clusters_excluding_noise | noise_points | silhouette | davies_bouldin |
|---|---|---|---|---|---|---|
| **0** | K-means | 2500 | 2 | 0 | 0.130576 | 3.479130 |
| **1** | Agglomerative | 1000 | 2 | 0 | 0.125571 | 4.942888 |
| **2** | DBSCAN | 1000 | 1 | 537 | NaN | NaN |
| **3** | Spectral | 500 | 2 | 0 | -0.046161 | 2.590577 |

## Model evaluation: what the metrics mean

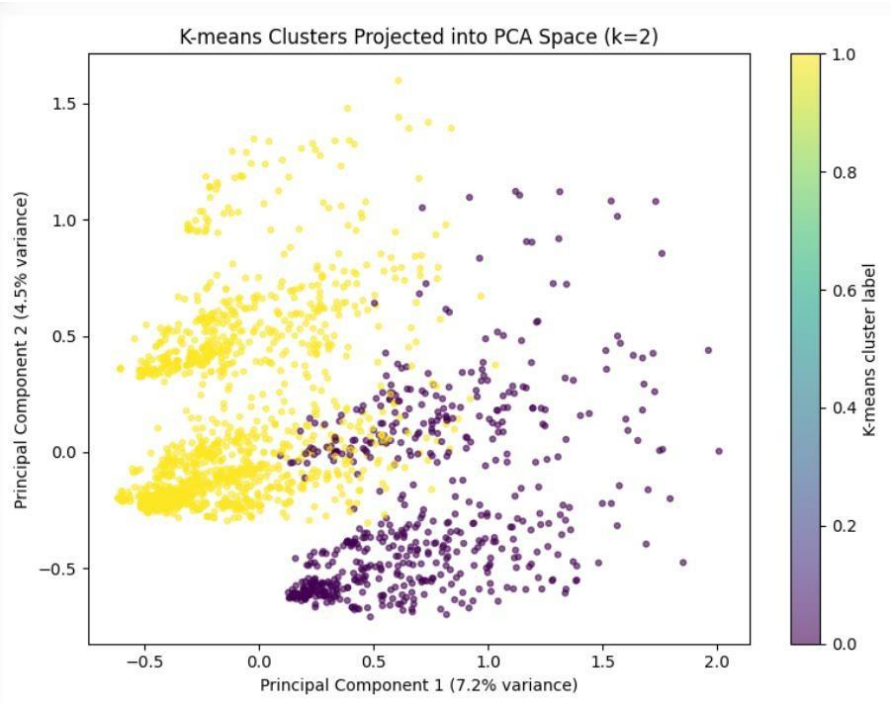
- **Silhouette coefficient:** ranges from −1 to 1; larger values generally mean better separation and cohesion.
- **Davies–Bouldin index:** lower values generally indicate more compact, separated clusters.

- **DBSCAN noise points:** rows labeled −1 are not assigned to a density cluster. If DBSCAN finds fewer than two non-noise clusters, its silhouette and Davies–Bouldin scores are left undefined (NaN), because those metrics would not be meaningful.

```
# 9. Visualize K-means labels in PCA space
# Refit K-means on the exact sample used for the PCA plot.
pca_km = KMeans(n_clusters=best_k, random_state=RANDOM_STATE, n_init=10)
pca_labels = pca_km.fit_predict(X_pca_input)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=pca_labels, s=12, alpha=0.6)
plt.title(f"K-means Clusters Projected into PCA Space (k={best_k})")
plt.xlabel(f"Principal Component 1 ({explained[0]:.1%} variance)")
plt.ylabel(f"Principal Component 2 ({explained[1]:.1%} variance)")
plt.colorbar(scatter, label="K-means cluster label")
plt.tight_layout()
plt.show()

cluster_sizes = pd.Series(pca_labels).value_counts().sort_index().rename("transaction_count")
display(cluster_sizes.to_frame())
print("Cluster sizes are counts within the PCA/K-means sample, not necessarily the full dataset.")
```




| | transaction_count |
|---|---|
| **0** | 618 |
| **1** | 1882 |

### Cluster interpretation

The colored PCA plot shows the K-means assignments for the PCA sample. Use the cluster-size table to identify whether the solution is balanced or dominated by a large cluster.

Because PCA compresses a high-dimensional binary matrix into two axes, apparent separation in this plot should be checked against the numeric metrics and product profiles.

```
# 10. Describe product profiles for the PCA/K-means clusters
# For each cluster, show the products with the highest within-cluster presence rate.
profile_rows = []
for cluster_id in sorted(np.unique(pca_labels)):
    mask = pca_labels == cluster_id
    rates = X_pca_input[mask].mean(axis=0)
    top_indices = np.argsort(rates)[::-1][:8]
    profile_rows.append({
        "cluster": int(cluster_id),
        "transactions_in_sample": int(mask.sum()),
        "top_products_by_presence": ", ".join(products[i] for i in top_indices if rates[i] > 0)
    })
cluster_profiles = pd.DataFrame(profile_rows)
display(cluster_profiles)
```

| | cluster | transactions_in_sample | top_products_by_presence |
|---|---|---|---|
| **0** | 0 | 618 | mineral water, spaghetti, eggs, milk, chocolat... |
| **1** | 1 | 1882 | french fries, eggs, chocolate, spaghetti, gree... |

**Business interpretation**

Cluster profiles can reveal which products are comparatively common in each group. Since common products may appear in several clusters, describe them as *relative profiles*, not exclusive customer types. Potential applications include testing product bundles, cross-selling recommendations, or arranging frequently co-occurring products nearby. These are hypotheses to test; this dataset alone cannot establish causal effects or customer-level behavior.

### 3. Statistical findings

Use the executed tables and figures above to report:

1. Total transactions and distinct products.
2. Typical transaction size (mean, median, quartiles, and spread).
3. The most frequently appearing products.
4. PCA variance retained by the first two components.
5. The selected K-means value of *k* and its silhouette/Davies–Bouldin scores.

6. How cluster sizes and product profiles differ.

## 4. Model evaluation

Compare silhouette and Davies–Bouldin values in the model comparison table. Prefer solutions with higher silhouette and lower Davies–Bouldin values, while also considering cluster count, noise, sample size, and interpretability. Scores should not be compared as if they came from identical samples because the algorithms were run on different-sized samples for practical runtime reasons. The best model is not necessarily the one with a single best metric; it should produce stable, interpretable groups useful to the business question.

### 5. BUSINESS RECOMMENDATIONS

1. **Identify Product Associations:** Use the frequent-product analysis to identify products that are commonly purchased together. This can help retailers understand customer purchasing patterns and plan suitable product combinations.
2. **Improve Product Bundling and Cross-Selling:** Use the clustering results to suggest possible product bundles and cross-selling opportunities. These suggestions should be tested before implementation, and their effectiveness should be measured using conversion rates or average basket value when appropriate sales and price data are available.
3. **Apply Suitable Clustering Techniques:** Since retail transactions contain binary and sparse product data, consider using Jaccard distance to measure the similarity between shopping baskets. Compare the clustering results with association-rule mining to identify meaningful relationships between products.
4. **Collect Additional Transaction Data:** In future studies, include invoice IDs, transaction dates, quantities, prices, and customer identifiers where available. This additional information would support further analysis of basket value, purchasing trends over time, repeat purchases, and customer-level buying patterns.

### 6. Limitations

- The file contains basket contents only; it has no customer, invoice, date, quantity, price, or revenue fields.
- Product names and transaction rows are treated as supplied; source-system data quality cannot be independently verified.
- Binary presence/absence ignores quantity and purchase value.
- PCA is only a two-dimensional approximation.
- Some clustering methods run on samples of different sizes for runtime reasons.
- DBSCAN parameters may need tuning; its metrics can be undefined when it finds fewer than two clusters after excluding noise.
- Clusters describe similarities in these baskets and should not be treated as causal customer segments.

## 7. CONCLUSION

This case study analyzed online retail transactions using statistical techniques, data visualization, Principal Component Analysis (PCA), and clustering algorithms, including K-means, Agglomerative Clustering, DBSCAN, and Spectral Clustering. The models were evaluated using silhouette scores, the Davies–Bouldin index, and cluster characteristics to identify similarities between shopping baskets.

Based on the experimental results, K-means with two clusters achieved the highest silhouette score among the tested K-means configurations. However, the score indicated weak separation between clusters, and the other clustering algorithms also showed limitations in identifying clearly distinct groups. The analysis revealed that mineral water was the most frequently occurring product and that transactions contained an average of 3.91 distinct products. These findings demonstrate how statistical analysis and clustering can help explore retail purchasing patterns and support product placement and promotional planning. Further improvements using association-rule mining and suitable basket-similarity measures are needed before applying the results to practical retail decisions.

## REFERENCES


1. 1. UCI Machine Learning Repository, Online Retail Dataset.
2. Scikit-learn documentation, *Classification Models and Model Evaluation Metrics*.
3. Pandas documentation, *Data Loading and Data Analysis*.
4. Matplotlib and Seaborn documentation, *Data Visualization*.